

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_30

30. FAQ

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter is a subset of the Q&A sessions of the chapters in this book. Many of these questions were not discussed in a previous chapter because the related patterns were not covered at that time. So, it is highly recommended that in addition to the following Q&As, you go through all the Q&A sessions in the book for a better understanding of these patterns.

30.1 Which design pattern do you like the most?

It depends on many factors such as the context, situation, demand, constraints, and so on. If you know about all of the patterns, you have more options to choose from.

30.2 Why should developers use design patterns?

They are reusable solutions for software design problems that appear repeatedly in real-world software development.

30.3 What is the difference between the Command and Memento patterns?

These two patterns can be used together, but their intents are different. Mementos store object states, but commands store the intended operations. Only the originator can store and retrieve necessary information from a memento. This memento is “opaque” to other objects. In the Command pattern, you do not see a similar kind of restriction. So, the

states of the commands are often readable, but this is not true for mementos.

You store mementos for future use. On the other hand, once a command is executed, its job is done. Now it is up to you whether you want to store it. If you do not want to support the undo functionality, you may not be interested in storing such a command.

Lastly, you can simply remember that to perform an undo operation, you can opt for either of these patterns. In this context, when you use the Memento pattern, you replace a state from a cache. But if you opt for the Command pattern, you re-execute the commands in the same order to reach a particular state.

30.4 What is the difference between the Facade pattern and the Builder pattern?

The Facade pattern aims to make a specific portion of code easier to use. It abstracts details away from the developer.

The Builder pattern separates the construction of an object from its representation. In Chapter [6](#), when I discuss the Builder pattern, you see that the director can call the same method (refer to the `instruct()` method in Demonstration 1) to create different types of vehicles. In other words, you can use the same construction process to create different types.

30.5 What is the difference between the Builder pattern and the Strategy pattern? They have similar UML representations.

First, you must take care of the intent. The Builder pattern falls into the category of creational patterns, and the Strategy pattern falls into the category of behavioral patterns. Their areas of focus are different. With the Builder pattern, you can use the same construction process to create different types, and with the Strategy pattern, you have the freedom to select an algorithm at runtime.

30.6 What is the difference between the Command pattern and the Interpreter pattern?

In the Command pattern, the commands are objects, but in the Interpreter pattern, the commands are sentences. In the Interpreter pattern, you can make your own rule for evaluation and build the syntax tree. If you deal with simple grammar, it is fine. But implementing the Interpreter pattern is tough when your grammar is complex. It is because the cost of building an interpreter can be a big concern for you.

30.7 What is the difference between the Chain of Responsibility pattern and the Observer pattern?

For the Observer pattern, all registered users will be notified or get requests (for the change in subject) in parallel, but for the Chain of Responsibility pattern, you may not reach the end of the chain, so all users do not need to handle the same scenario. The request can be processed much earlier by a handler object that is placed at the beginning of the chain. I suggest you refer to Q&A [17.7](#) to understand it clearly.

30.8 What is the difference between the Chain of Responsibility pattern and the Decorator pattern?

They are not the same at all, but you may feel that they are similar in their structures. Similar to the previous difference, at a given point in time, in the Chain of Responsibility pattern, only one class handles the request, but in the case of the Decorator pattern, all classes handle the request. You must remember that decorators are effective in the context of adding and removing responsibilities only, and if you can combine the Decorator pattern with the Single Responsibility Principle (SRP), you can add (or remove) a single responsibility at runtime.

30.9 What is the difference between the Mediator pattern and the Observer pattern?

The GoF says this: “These are competing patterns. The difference between them is that the Observer distributes communication by introducing observer and subject objects, whereas a mediator object encapsulates the communication between other objects.”

Normally you see the use of observers for one-to-many communications. So, this type of communication is simple to understand and often used as a Publisher-Subscriber (Pub-Sub) model.

On the other hand, mediators are common for many-to-many communications. If any of the participants change, all other participants are notified. So, compared to the Observer pattern, this kind of communication is harder to implement.

The GoF also found that you may face fewer challenges when making reusable observers and subjects than when making reusable mediators, but regarding the flow of communication, the Mediator pattern scores higher than the Observer pattern.

30.10 Which one do you prefer: a singleton class or a static class?

In Chapter [2](#) (Q&A [2.4](#)), I told you that at the time of this writing a top-level static class is not possible in Java. Although it is possible in a different programming language such as C#, you need to obey the language-specific restrictions for using this construct.

So, the general answer is, it depends on many factors. But the key take-away is you can create objects of a singleton class, which is not possible with a static class. So, the concepts of inheritance and polymorphism can be implemented with a singleton class. Also, you can pass a singleton instance as a method parameter.

30.11 How can you distinguish between proxies and adapters?

Proxies work on similar interfaces as their subjects. Adapters work on different interfaces (to the objects they adapt).

30.12 How are proxies different from decorators?

There are different types of proxies, and they vary as per their implementations. So, some of these implementations may be close to decorators. For example, a protection proxy might be implemented like a decorator.

But you must remember that decorators focus on adding responsibilities, while proxies focus on controlling the access to an object.

30.13 How are mediators different from facades?

In general, both simplify a complex system. In the case of a Mediator pattern, a two-way connection exists between a mediator and the internal subsystems, whereas in the case of a Facade pattern, in general, you provide a one-way connection (the subsystems do not know about facades).

30.14 Is there any relation between the Flyweight pattern and the State pattern?

The GoF says that the Flyweight pattern can help you to decide when and how to share the state objects.

30.15 What are the similarities among the Simple Factory, Factory Method, and Abstract Factory design patterns?

All of them encapsulate object creation. They suggest that you try to code to the abstraction (interface) but not to the concrete classes. In simple words, each of these factories promotes loose coupling by reducing the dependencies on concrete classes.

30.16 What are the differences among the Simple Factory, Factory Method, and Abstract Factory design patterns?

This is an important question that you may face in various job interviews. So, first, I refer you to Q&A [3.6](#) in Chapter [3](#) and Q&A [4.3](#) in Chapter [4](#). If needed, you can go through all the Q&A sessions in Chapter [3](#) and Chapter [4](#).

30.17 How can you distinguish the Singleton pattern from the Factory Method pattern?

The Singleton pattern ensures you get a unique instance each time. It also restricts the creation of additional instances.

But the Factory Method pattern does not say that you will get a unique instance only. Most often this pattern is used to create as many instances as you want, and the instances are not necessarily unique. These newly typed instances may implement a common parent class. (Just remember that the Factory method lets a class defer instantiation to subclasses, per the GoF definition.)

30.18 How does the Template Method pattern differ from the Strategy pattern?

In the Strategy pattern, you can vary the entire algorithm using delegation. On the other hand, using the Template Method pattern you vary only certain step(s) in an algorithm using inheritance, but the overall flow of the algorithm is unchanged.

30.19 How can you distinguish the Visitor pattern from the Strategy pattern?

In the Strategy pattern, each subclass uses a different algorithm to solve a common problem. But in the case of the Visitor design pattern, each visitor subclass may provide different functionalities.

30.20 How are null objects different from proxies?

In general, proxies act on real objects at some point in time and they may also provide some behavior. But, in general, a null object does not do any such operation.

30.21 How can you distinguish the Interpreter pattern from the Visitor pattern?

In the Interpreter pattern, you represent a simple grammar as an object structure, but in the case of the Visitor pattern, you define some specific operations that you want to use on an object structure. In addition to this, an interpreter has direct access to the behaviors that are needed but in the case of the Visitor pattern, you need special functionalities (similar to an observer) to access them.

30.22 How can you distinguish the Flyweight pattern from the Object Pool pattern?

In the case of the Flyweight pattern, flyweights can have intrinsic and extrinsic states. So, you can say that if a flyweight has both states, its states are divided and the client needs to pass part of the state to it. Also, in general, the client does not change the intrinsic state because it is shared.

I did not discuss the Object Pool pattern in this book. But if you know this pattern already, you will notice that the Object Pool pattern does not store any part of the state outside; all state information is stored/encapsulated inside the pooled object. Also, clients can change the state of a pooled object.

30.23 How are libraries (or frameworks) similar (or different) from design patterns?

They are not considered design patterns. They provide the implementations that you can use directly in your application. But they can use the concept of the patterns in those implementations.

30.24 How can you compare decorators and adapters?

The common theme between object adapters and decorators is that they both promote object composition, but there are many differences.

You use decorators to enhance an object's behavior. You do not alter the original interface, so you can decorate an object with a decorator, and then you can again decorate this decorated object with another decorator. This kind of recursive decoration/composition is easy in the Decorator pattern.

But adapters are useful for incompatible interfaces. There you adapt the original behavior, but do not impose any change in this behavior. In this pattern, it is not easy to implement a recursive composition.

30.25 How can you distinguish the Prototype pattern from the Singleton pattern?

In the case of the Singleton pattern, you always work on the same instance. But in the case of the Prototype pattern, you work on a copied/cloned instance. The state(s) between two copied instances may vary, but it is not possible when you work with the Singleton pattern because every time you work on the exact same instance.

30.26 How can you distinguish between a Chain of Responsibility pattern and a Command pattern?

In the Chain of Responsibility pattern, handlers pass requests one after another, but there is no guarantee that they will be fully processed by any of them.

In the case of the Command pattern, you do not see similar kinds of passing requests. But you can track commands to provide undo operations, which is not seen with the Chain of Responsibility pattern.

Previous chapter

< [29. Anti-Patterns](#)

Next chapter

[Back Matter](#) >